

Structure of the Project:

Cifrovoj dvojnik elektriicheskoj seti

Structura proekta:

Cifrovoj dvojnik elektriicheskoj seti

Technical documentation for z.AI

April 2026

Soderzanie

1. Obzor proekta	4
1.1 Naznachenie	4
1.2 Technological Stack	4
2. Poryadok zapuska i inicializacii processov	4
2.1 Zapusk servera (npm run dev)	4
2.2 Inicializacia Prisma i bazy dannyh	4
2.3 Poryadok zagruzki stranicy	5
3. Structura katalogov i fajlov	5
3.1 Kornevye konfiguracionnye fajly	5
3.2 Katalog src/app/ - Entry Point (Next.js App Router)	6
3.2.1 src/app/layout.tsx	6
3.2.2 src/app/page.tsx	6
3.2.3 src/app/globals.css	6
3.3 Katalog src/app/api/ - Server API Routes	6
3.4 Katalog src/lib/ - Business Logic	7
3.4.1 src/lib/db.ts	7
3.4.2 src/lib/services/import.service.ts	7
3.4.3 src/lib/services/validation.service.ts	7
3.4.4 src/lib/data/references.ts	7
3.4.5 Calculation Modules (src/lib/calculations/)	8
3.4.6 src/lib/utills/id-generator.ts	8
3.5 Katalog src/types/index.ts	9
3.6 Katalog src/components/ - UI Components	9
3.6.1 NetworkGraph.tsx (Client Component)	9
3.6.2 ElementDetails.tsx (Client Component)	9

3.6.3 ValidationPanel.tsx (Client Component)	10
3.6.4 src/components/ui/ (48 files)	10
3.7 Katalog src/hooks/	10
3.8 Katalog prisma/	10
3.9 Katalog public/	10
3.10 Katalog upload/	10
3.11 Katalog download/	10
4. Arhitektura dannyh	10
4.1 Prisma Schema (6 Layers)	10
4.2 Data Flow	11
5. API-Endpointy - Detailnyj spravochnik	11
6. Pravila validacii	12

1. Obzor proekta

1.1 Naznachenie

Project "Cifrovoj dvojnik elektriicheskoy seti" (Network Digital Twin) is designed for modeling, visualization and validation of low-voltage electrical networks (0.4 kV). The system allows importing network topology data from Excel files, automatically calculating electrical parameters (cable line impedances, short-circuit currents, voltage drops), performing compliance checks against PUE requirements (Rules for Electrical Installation), and displaying the network as an interactive mnemonic diagram. The project is implemented as a web application based on Next.js 16 with server-side API on Prisma ORM and client-side SVG/React visualization.

1.2 Technological Stack

- Framework: Next.js 16.1.3 (App Router, Turbopack)
- Language: TypeScript 5
- Styling: Tailwind CSS 4 + shadcn/ui (new-york style)
- Database: SQLite via Prisma ORM 6.19.2
- Build: standalone output (production: bun server.js)
- Additional libraries: `xlsx` (Excel import), `lucide-react` (icons), `next-themes` (theme), `recharts` (charts), `framer-motion` (animations), `zustand` (state management), `react-resizable-panels`

2. Poryadok zapuska i inicializacii processov

2.1 Zapusk servera (npm run dev)

When executing `npm run dev` the following sequence occurs:

Step 1. Node.js reads `package.json` and launches `next dev -p 3000`. Next.js 16 initializes the Turbopack compiler and begins monitoring the file system.

Step 2. Next.js loads configuration from `next.config.ts`: `output="standalone"` mode, TypeScript error suppression during build, `reactStrictMode=false`.

Step 3. Next.js loads environment variables from the `.env` file:

`DATABASE_URL=file:/home/z/my-project/db/custom.db`.

Step 4. Prisma ORM initializes on the first API request (lazy initialization via singleton in `src/lib/db.ts`).

2.2 Inicializacia Prisma i bazy dannyh

The file `src/lib/db.ts` creates a singleton `PrismaClient` instance with SQL query logging (`log: ['query']`). In dev mode the client is stored in `globalThis` to prevent multiple connections during hot-reload. The SQLite database is located at the path from `DATABASE_URL`. The schema is defined in `prisma/schema.prisma` and contains 20+ models: `Element`, `Connection`, `Network`, `Device`, `DeviceState`,

Protection, AtsLogic, Scenario, Command, PowerFlow, ShortCircuit, Measurement, Alarm, AlarmRule, Tariff, Maintenance, EventLog, CableReference, BreakerReference, TransformerReference, ValidationRule, ValidationResult.

2.3 Poryadok zagruzki stranicy

On GET / request, Next.js executes:

1. Loading `src/app/layout.tsx` (Server Component):

- Registering Geist and Geist_Mono fonts from Google Fonts
- Importing globals.css (Tailwind CSS 4 with CSS variables for light/dark themes)
- Wrapping children in ThemeProvider (next-themes) and Toaster (shadcn)
- Setting metadata: title="Cifrovoj dvojn timer elektricheskoy seti"

2. Loading `src/app/page.tsx` (Client Component, 'use client'):

- Mount component -> setMounted(true)
- useEffect calls loadNetworkData() and loadValidationResults()
- loadNetworkData() makes fetch('/api/network') -> receives GraphData (nodes + edges)
- loadValidationResults() makes fetch('/api/validation') -> receives ValidationIssue array
- Data stored in React state (graphData, issues, stats, validationStats)

3. Structura katalogov i fajlov

3.1 Kornevye konfiguracionnye fajly

File	Purpose
package.json	Dependencies and project scripts (884 packages)
next.config.ts	Next.js configuration: standalone output, TypeScript relaxed
tsconfig.json	TypeScript configuration: target ES2017, path alias @/ -> ./src/*
components.json	shadcn/ui configuration: new-york style, lucide icons
tailwind.config.ts	Tailwind CSS configuration
postcss.config.mjs	PostCSS with @tailwindcss/postcss
eslint.config.mjs	ESLint configuration
.env	DATABASE_URL=file:/home/z/my-project/db/custom.db
Caddyfile	Caddy reverse proxy configuration

Table 1. Root configuration files

3.2 Katalog src/app/ - Entry Point (Next.js App Router)

3.2.1 src/app/layout.tsx

Root layout (Server Component). Initializes: Geist Sans and Geist Mono fonts, CSS theme (globals.css), ThemeProvider (light/dark via class attribute), Toaster for notifications.

3.2.2 src/app/page.tsx

Main page (Client Component). Contains: State (graphData, issues, stats, validationStats, selectedNode, selectedEdge, zoom, searchTerm, isLoading, isImporting, isValidating). Functions: loadNetworkData(), loadValidationResults(), handleImport(), handleValidate(), handleNodeClick(), handleEdgeClick(). UI: header (search, zoom, statistics, import/validation/theme buttons), main (NetworkGraph + ElementDetails + ValidationPanel).

3.2.3 src/app/globals.css

Tailwind CSS 4 with tw-animate-css import. Defines CSS variables for light/dark themes via oklch colors: background, foreground, card, primary, secondary, muted, accent, destructive, border, input, ring, chart-1..5, sidebar.

3.3 Katalog src/app/api/ - Server API Routes

Route	Methods	Purpose
/api/route.ts	GET	Health-check: returns {message: "Hello, world!"}
/api/network/route.ts	GET	Returns full network topology (GraphData: nodes + edges)
/api/elements/route.ts	GET, POST, PUT, DELETE	CRUD of network elements (Element) with devices
/api/devices/route.ts	GET, POST, PUT, DELETE	CRUD of devices (Device)
/api/connections/route.ts	GET, POST, PUT, DELETE	CRUD of connections (Connection) with auto impedance calculation
/api/validation/route.ts	GET, POST	GET: list of issues; POST: run validation
/api/import/route.ts	POST	Import data from Excel file upload/input.xlsx
/api/references/route.ts	GET	Reference data: cables, breakers, transformers

Table 2. API Routes

API Request Processing Sequence:

1. Incoming HTTP request -> Next.js calls the corresponding route.ts
2. route.ts imports **db** from **@/lib/db** -> PrismaClient singleton initializes (if not yet)
3. Prisma executes SQL queries to SQLite
4. Result is converted to JSON and returned via NextResponse.json()

3.4 Katalog src/lib/ - Business Logic

3.4.1 src/lib/db.ts

Singleton PrismaClient. Exports **db** - a single instance of the SQLite connection with query logging.

3.4.2 src/lib/services/import.service.ts

Excel data import service. Main function: **importFromExcel()**.

Workflow:

1. **resetCounters()** - reset ID generator counters
2. Check existence of upload/input.xlsx
3. Clear all database tables (**clearDatabase**): **ValidationResult** -> **Measurement** -> **DeviceState** -> **Protection** -> **AtsLogic** -> **Command** -> **PowerFlow** -> **ShortCircuit** -> **Network** -> **Connection** -> **Device** -> **Element**
4. Read "Elements" sheet to build type map (**elementTypeMap**)
5. Process "Networkall" sheet: create **Element** + **Device** for each unique name + **Connection** between pairs
6. Process additional sheets: **Sources**, **Cabinets**, **Loads**, **Breakers**, **Connections/Cables**
7. If no data - create demo data (**createDemoData**)
8. Calculate node positions for visualization (**calculateNodePositions**)

3.4.3 src/lib/services/validation.service.ts

Network validation service. Main functions: **runValidation()**, **getValidationIssues()**, **getValidationStats()**.

runValidation() workflow:

1. Clear previous results (**ValidationResult.deleteMany**)
2. Execute 4 checks in parallel:
 - **checkCableCapacity()** (**CABLE_001**): $I_{nom.breaker} \leq I_{admissible.cable}$
 - **checkVoltageDrop()** (**VOLTAGE_001**): voltage drop $\leq 4\%$
 - **checkProtectionSensitivity()** (**PROT_001**): $I_{sc.end} \geq 3 * I_{nom.breaker}$ (**CRITICAL**)
 - **checkSelectivity()** (**SEL_001**): $I_{nom.incomer} \geq I_{nom.outgoing}$
3. Create/update **ValidationRule** in database
4. Save results **ValidationResult**

Auxiliary functions:

- **findPathToSource(elementId)** - graph traversal from element to **SOURCE** (max 50 iterations)
- **getAllDownstreamElements(sourceId)** - BFS from source downstream

3.4.4 src/lib/data/references.ts

Equipment reference data (static):

- CABLE_REFERENCES: 29 cables (14 AVVG aluminum + 15 VVG copper) with PUE parameters: cross-section, cores, R Ohm/km, X Ohm/km, I air, I ground
- BREAKER_REFERENCES: 7 breaker models (VA-47-29, VA-47-100, VA-55-41, VA-55-43, S203, NSX100, NSX250) with parameters: In, poles, U_nom, breaking capacity
- TRANSFORMER_REFERENCES: 6 transformers (TM-250/10 ... TM-2500/10) with parameters: S kVA, U_k%, short-circuit/no-load losses

Search functions: findCableReference(), findBreakerReference(), findTransformerReference(), getCableCurrentCapacity(), getReferencesData()

3.4.5 Calculation Modules (src/lib/calculations/)

impedance.ts - Cable line impedance calculation:

- calculateCableImpedance(length, wireSize, material, wireType) -> {r, x, z} by formulas $R=r_0 \cdot L/S$, $X=x_0 \cdot L/1000$, $Z=\sqrt{R^2+X^2}$
- calculateCableImpedanceFromReference(length, wireType, wireSize) -> from reference data
- calculateCableCurrentCapacity(wireType, wireSize, installationMethod) -> admissible current
- adjustResistanceForTemperature(r20, temperature, alpha=0.004) -> temperature correction
- calculateCablePowerLoss(current, r) -> losses: $dP = 3 \cdot I^2 \cdot R$

shortCircuit.ts - Short-circuit current calculation:

- calculateShortCircuit(zSystem, zCable, voltage) -> {ik3, ik1, ik2, zk}
- calculateShortCircuitWithTransformer(S, uk%, zCable, U=400) -> with transformer impedance
- calculatePeakShortCircuitCurrent(ik3, kud=1.8) -> peak current: $i_{ud} = \sqrt{2} \cdot K_{ud} \cdot I_{k3}$
- checkProtectionSensitivity(ikMin, inBreaker, ratio=3) -> protection sensitivity check
- calculateMinShortCircuitCurrent(zSource, zLine, voltage) -> minimum SC current with heating factor (1.5)

voltageDrop.ts - Voltage drop calculation:

- calculateVoltageDrop(P, Q, R, X, U_kV) -> $dU\% = (P \cdot R + Q \cdot X) / (10 \cdot U^2)$
- calculateVoltageDropByCurrent(I, R, X, cosPhi, U) -> $dU\% = \sqrt{3} \cdot I \cdot (R \cdot \cos\phi + X \cdot \sin\phi) / U \cdot 100$
- calculateVoltageDropByLength(I, L, S, material, cosPhi, U) -> full calculation by length
- isVoltageDropAcceptable(dU%, max=4%) -> check
- calculateMinWireSizeForVoltageDrop(P, L, material, maxDrop%, U, cosPhi) -> minimum cross-section

3.4.6 src/lib/utls/id-generator.ts

ID generation functions:

- generateElementId(type, prefix?, name?) -> ID format: SRC_TP01, CAB_XXX, LOAD_XXX, BUS_XX, MTR_XXX, QF_XXX
- generateDeviceId(type, parentId?) -> DEV_S001, DEV_B001, DEV_L001 etc.
- generateConnectionId(fromId?, toId?) -> CONN_{from}_{to}_{counter}
- generateValidationId(ruleCode, elementId?) -> VAL_{code}_{elementId}_{timestamp}
- resetCounters() - reset all counters

- `parseElementType(id)` -> determine type by ID prefix
`src/lib/utils.ts` - Utility `cn()` for CSS class merging (`clsx` + `tailwind-merge`).

3.5 Katalog `src/types/index.ts`

TypeScript type definitions for the project:

- Element types: `ElementType` = SOURCE | CABINET | LOAD | BUS | METER | BREAKER | JUNCTION
- Device types: `DeviceType` = SOURCE | BREAKER | LOAD | METER | ATS | SWITCH | TRANSFORMER
- Connection types: `ConnectionType` = CABLE | BUSBAR | JUMPER
- Graph: `GraphNode`, `GraphEdge`, `GraphData`
- Validation: `ValidationStatus`, `ValidationCategory`, `ValidationResultData`, `ValidationIssue`, `ValidationRule`, `ValidationContext`
- Calculations: `ImpedanceResult`, `ShortCircuitResult`
- References: `CableReferenceData`, `BreakerReferenceData`, `TransformerReferenceData`
- API: `NetworkStats`, `ImportResponse`, `ApiResponse`
- Forms: `AddElementFormData`

3.6 Katalog `src/components/` - UI Components

3.6.1 `NetworkGraph.tsx` (Client Component)

SVG network topology visualization. Key features:

- `layoutData` (`useMemo`): BFS algorithm for hierarchical layout (Source top -> Load bottom)
- `bounds` (`useMemo`): automatic graph boundary detection
- `Pan/Drag`: mouse handlers for canvas movement
- `Zoom`: mouse wheel handler + external control via props
- SVG rendering: nodes (140x70 blocks with type color bar, icon, name, ID, LIVE/DEAD status, problem indicator), edges (lines with cable labels, status indicators)
- `Tooltip`: info panel on node/edge hover
- Color scheme: adapts to theme (light/dark). Element types have unique colors: source (yellow-green gradient), bus/junction/cabinet (copper), breaker/load (green), meter (blue)

3.6.2 `ElementDetails.tsx` (Client Component)

Selected element/connection details panel. Two modes:

- Node mode: Badge with type, ON/OFF and LIVE/DEAD statuses, problem indicator, device parameters (I_{nom} , P, Q, $\cos \phi$, S, model), validation results, device list.
- Connection mode: cable parameters (brand, cross-section, cores, material, length, $I_{admissible}$, installation), electrical parameters (R, X, Z), calculated parameters (voltage drop, short-circuit current, load current).

3.6.3 ValidationPanel.tsx (Client Component)

Validation issues panel. Two display modes:

- `compact=true` (default): mini-panel in corner with issue count, expandable list of critical issues (up to 3) and re-check button.
- `compact=false`: full view with statistics and button.

3.6.4 src/components/ui/ (48 files)

shadcn/ui component library: accordion, alert-dialog, alert, aspect-ratio, avatar, badge, breadcrumb, button, calendar, card, carousel, chart, checkbox, collapsible, command, context-menu, dialog, drawer, dropdown-menu, form, hover-card, input, input-otp, label, menubar, navigation-menu, pagination, popover, progress, radio-group, resizable, scroll-area, select, separator, sheet, sidebar, skeleton, slider, sonner, switch, table, tabs, textarea, toast, toaster, toggle, toggle-group, tooltip.

3.7 Katalog src/hooks/

- `use-mobile.ts` - mobile device detection (shadcn/ui)
- `use-toast.ts` - toast notification management hook (shadcn/ui)

3.8 Katalog prisma/

- `schema.prisma` - full database schema (20+ models, SQLite, 637 lines)
- `powergrid.db` - SQLite database file (512 KB, contains network data)

3.9 Katalog public/

Static images: `breaker.jpg`, `load.jpg`, `meter.jpg`, `source.jpg`. Icons: `logo.svg`, `file.svg`, `globe.svg`, `next.svg`, `vercel.svg`, `window.svg`. `robots.txt`.

3.10 Katalog upload/

`input.xlsx` - source Excel file for network topology import. Additional files: network element images.

3.11 Katalog download/

- Documentation: `ARCHITECTURE.md`, `README.md`
- Data export: `chatgpt_chat.json`, `deepseek_chat.json`
- Models: `network_model.xlsx`, `network_model_v2..v10`
- Scripts: `network_model_builder.py`, `network_model_v10_industrial.py`
- Instructions: Installation guide (DOCX)

4. Arhitektura dannyh

4.1 Prisma Schema (6 Layers)

Layer	Models	Description
Layer 1 - Topology	Element, Connection, Network	Graph nodes, edges, and link scenarios
Layer 2 - Equipment	Device, DeviceState, DeviceType	Equipment instances, states, and types
Layer 3 - Control	AtsLogic, Protection, Scenario, Command	ATS logic, protection relays, scenarios, commands
Layer 4 - Calculations	PowerFlow, ShortCircuit, LoadProfile	Power flow analysis, SC calculations, load profiles
Layer 5 - Measurements	Measurement, Alarm, AlarmRules	Measurement data, alarms and alarm rules
Layer 6 - Maintenance	Tariff, Maintenance, EventLog	Tariffs, maintenance scheduling, event logging
References	CableReference, BreakerReference, TransformerReference, ValidationRule	Static equipment reference data and validation rules
Validation	ValidationResult	Validation check results storage

Table 3. Prisma Schema Layers

4.2 Data Flow

Process	Data Flow
Import	Excel -> import.service -> Prisma -> SQLite
Visualization	SQLite -> Prisma -> /api/network -> GraphData -> NetworkGraph (SVG)
Validation	SQLite -> Prisma -> validation.service -> /api/validation -> ValidationPanel
Editing	UI -> /api/elements devices connections -> Prisma -> SQLite

Table 4. Data Flow

5. API-Endpointy - Detailnyj spravochnik

Endpoint	Method	Request Body	Response	Description
/api/	GET	None	{message: "Hello, world!"}	Health check
/api/network	GET	None	GraphData {nodes, edges}	Full network topology
/api/elements	GET	None	Element[]	List all elements

Endpoint	Method	Request Body	Response	Description
/api/elements	POST	AddElementFormData	Element	Create element
/api/elements/[id]	PUT	Partial	Element	Update element
/api/elements/[id]	DELETE	None	{success: true}	Delete element
/api/devices	GET	None	Device[]	List all devices
/api/devices	POST	DeviceCreateInput	Device	Create device
/api/devices/[id]	PUT/DELETE	Partial / None	Device / {success}	Update/Delete device
/api/connections	GET	None	Connection[]	List all connections
/api/connections	POST	ConnectionCreateInput	Connection	Create connection (auto-calculates impedance)
/api/connections/[id]	PUT/DELETE	Partial / None	Connection / {success}	Update/Delete connection
/api/validation	GET	None	ValidationIssue[]	List validation issues
/api/validation	POST	None	{issues, stats}	Run validation checks
/api/import	POST	FormData (xlsx)	ImportResponse	Import Excel data
/api/references	GET	None	{cables, breakers, transformers}	Equipment reference data

Table 5. API Endpoints Detailed Reference

6. Pravila validacii

Code	Name	Severity	Category	Description	Formula
CABLE_001	Cable capacity check	ERROR	Capacity	Breaker rated current must not exceed cable admissible current	$I_{nom.breaker} \leq I_{adm.cable}$
VOLTAGE_001	Voltage drop check	WARNING	Voltage	Voltage drop along the cable must not exceed 4% of rated voltage	$dU\% \leq 4\%$
PROT_001	Protection sensitivity check	CRITICAL	Protection	Minimum short-circuit current at the end of the line must be at least 3 times the breaker rated current for reliable protection operation	$I_{sc.min} \geq 3 * I_{nom.breaker}$

Code	Name	Severity	Category	Description	Formula
SEL_001	Selectivity check	WARNING	Selectivity	Incomer breaker rated current must be greater than or equal to outgoing breaker rated current for proper selective coordination	$I_{nom.incomer} \geq I_{nom.outgoing}$

Table 6. Validation Rules